

Web Crawler Project

Demo: demo_video.mp4 (included in submission)

This is a web crawler and search engine built with Python, Flask, and SQLite. It crawls web pages using BFS traversal with back pressure controls, and it provides real-time search over indexed content. The crawling logic uses Python standard library modules and does not rely on BeautifulSoup, Scrapy, or requests. SQLite runs in WAL mode, so search can still work while crawls are active.

The project was built using Claude Code. It runs entirely on localhost and starts with `python app.py`. For assignment compatibility, the project also writes raw storage data to `data/storage/p.data` and supports the route `/search?query=<word>&sortBy;=relevance`.

Pages

1. Crawler (Home Page)

Users create new crawl jobs by entering an origin URL and depth `k`. There are also optional parameters for hit rate, max URLs to crawl, and queue capacity. Clicking Start Crawl creates a new thread and begins BFS traversal from the origin URL. The page also shows live system stats such as total jobs, pages indexed, unique words, and active crawlers. Recent crawl jobs are listed below with links to each job's status page.

2. Crawler Status

This page shows live monitoring for each crawl. It polls the server every 2 seconds and displays the current status, pages crawled, queue size, and max depth. A details area shows the crawler ID, origin URL, hit rate, max URLs, and queue capacity. A live logs section shows info, warning, and error messages as they arrive. Users can pause, resume, or stop the crawl from this page. When the queue fills up, back pressure warnings appear in the logs.

3. Search

Users type one or more keywords and get results as triples: `relevant_url`, `origin_url`, and `depth`. Results also show a relevance score and the matched words. Search supports sorting by relevance, frequency, or depth. There is also an I'm Feeling Lucky button that picks a random indexed word. Pagination is available for large result sets.

Crawler Job (Major Component)

This is the core of the project. It takes a URL and depth, along with settings for hit rate, max URLs, and queue capacity.

When a crawl starts, a unique ID is generated from the epoch timestamp and thread ID, using the format `[EpochTime_ThreadID]`. The crawler uses a bounded queue `Queue(maxsize=max_queue)` for BFS traversal. When the queue is full, new URLs are skipped and a warning is logged.

For each URL in the queue, the crawler:

1. Checks if the URL was already visited by using an in-memory set and a database lookup.
2. Waits based on the rate limit in short chunks, so pause and stop actions stay responsive.

3. Fetches the HTML with `urllib.request`, using SSL fallback, a 10 second timeout, and a 10 MB size limit.
4. Parses the HTML with `html.parser.HTMLParser` to extract the title, visible text, links, and word frequencies.
5. Stores page metadata in the `pages` table and word to URL mappings in the `word_index` table.
6. Adds discovered links at depth + 1 to the queue if they are within `max_depth` and the queue is not full.

The crawler supports pause, resume, and stop through thread-safe event objects. When a crawl is stopped, the remaining queue is saved to the database so the job can continue later. If a crawl fails before indexing any page, the job is marked as an error instead of showing a misleading completed state.

Search (Minor Component)

When a user types a query, each word is normalized and looked up in the `word_index` table. The search logic follows the assignment-compatible scoring rules and groups results by URL.

- Exact matches use the formula $(\text{frequency} * 10) + 1000 - (\text{depth} * 5)$.
- Prefix fallback is used only when there is no exact match for that query word.
- Results can be sorted by relevance, frequency, or depth.
- Because SQLite runs in WAL mode, search can run while crawls are still writing new data.

The assignment-friendly `/search` route makes it easy to compare raw storage lines in `p.data` with live search output.

Technical Details

Storage: SQLite with WAL mode, B-tree indexes on word lookups, and raw storage export to `data/storage/p.data`.

Concurrency: Thread-per-crawl model. Multiple crawl jobs can run at the same time.

Back Pressure: Bounded queue, configurable hit rate, and max URL cap.

Resumability: Queue state is saved on stop. Visited URLs and crawl state are loaded from the database on resume.

Testing: 55 unit tests cover the HTML parser, search service, and crawler job.

Dependencies: Flask is the main external framework dependency. Crawling logic uses Python standard library modules.

Repository: <https://github.com/deniz-cangoz/crawler>

Run locally with: `python app.py`, then open `http://localhost:3600`